Comprehensive Study Guide on Regular Languages, Regular Expressions, NFA, DFA, and Regular Grammars

1. Overview: What Makes a Language Regular

A Regular Language is any language that can be described using a regular expression, accepted by a finite automaton (DFA or NFA), or generated by a regular grammar.

Equivalences: - Regular Expression ↔ NFA ↔ DFA ↔ Regular Grammar - All represent the same set of regular languages.

2. Regular Expressions

Regular expressions describe patterns of strings using symbols and operators, providing a compact way to represent regular languages.

Basic Operators:

- Union (+): $L(r1 + r2) = L(r1) \cup L(r2)$ - Concatenation (·): $L(r1 \cdot r2) = \{ xy \mid x \in L(r1), y \in L(r2) \}$ - Kleene Star (*): $L(r1^*)$ = zero or more repetitions of $L(r1)$

Example: For $\Sigma = \{a, b\}$, $r = (a + b)^* (a + bb)$ represents all strings ending in 'a' or 'bb'.

3. Converting Regular Expressions to NFA

Theorem: For every regular expression r, there exists an NFA that accepts L(r).

Construction steps: 1. Build NFAs for base cases (Ø, λ, a ∈ Σ) 2. Combine them using +, concatenation, and * 3. The resulting NFA accepts L(r)

Example: r = (a + bb)*(ba* + λ)

Represents strings with any number of 'a' or 'bb', optionally followed by 'b' and any number of 'a'.

4. Nondeterministic Finite Automata (NFA)

An NFA allows multiple transitions for a single input, including λ-transitions. Formal definition: M = (Q, Σ, δ, q0, F), where δ: Q × (Σ ∪ {λ}) → 2^Q.

Acceptance: A string is accepted if any sequence of transitions ends in a final state.

Example: δ(q0, a) = {q1}, δ(q0, b) = {q2}, δ(q1, a) = {q1}, δ(q1, λ) = {q2}, final = q1. Language accepted: a+.

5. Conversion: NFA → DFA (Subset Construction)

Each DFA state represents a set of NFA states. The DFA accepts the same language as the NFA.

Algorithm: 1. Compute ε-closures of NFA states 2. Start with ε-closure(q0) 3. For each DFA state and input symbol, find reachable NFA states 4. Mark any DFA state containing an NFA final state as accepting.

Example (Your NFA): q0 -a→ q1, q0 -b→ q2, q1 -a→ q1, q1 -ε→ q2, final = q1. Equivalent DFA: S(start) -a→ A(accept), A -a→ A, all b→ D(dead). Language = a+.

6. Regular Grammars

A Regular Grammar is either Right-Linear or Left-Linear, equivalent to regular languages.

Right-linear example: S → aA | λ, A → aA | bbB, B → bC, C → S → Language: (a + bbb)*

Left-linear example: S → Aa | λ, A → Aa | Bbb, B → Cb, C → S → Language: (bbba+)*

7. Summary of Equivalences

Formalism	Notation	Recognizes
Regular Expression	r	Pattern of strings using +, ·, *
NFA	M = (Q, Σ, δ, q0, F)	Non-deterministic finite automaton
DFA	M = (Q, Σ, δ, q0, F)	Deterministic finite automaton
Regular Grammar	G = (V, T, S, P)	Right- or left-linear grammar

8. Quick Formula & Conversion Sheet

Regular Expression Rules: - L(r1 + r2) = L(r1) ∪ L(r2) - L(r1 r2) = concatenation - L(r1*) = repetition (zero or more times) Conversions: - RE → NFA: Thompson's Construction - NFA → DFA: Subset Construction - NFA ↔ Regular Grammar: Replace states with variables and transitions with productions Language Equivalence: Regular Expressions = NFAs = DFAs = Regular Grammars

End of Study Guide — Prepared for comprehensive review and quick exam mastery.